ASSIGNMENT – 6

NAME – BHARTI VERMA

ROLL NO.- 102303575

Question: 1 Device Query

```
bharti_vermaa@Bharti:~/UCS645/LAB6$ ./deviceQuery
Number of CUDA Devices: 1

----- Device 0 -----
Device Name: NVIDIA GeForce GTX 1650
Compute Capability: 7.5
Total Global Memory: 4095 MB
Shared Memory per Block: 48 KB
Constant Memory: 64 KB
Warp Size: 32
Max Threads per Block: 1024
Max Threads Dimension: (1024, 1024, 64)
Max Grid Size: (2147483647, 65535, 65535)
Multiprocessor Count: 14
Concurrent Kernels: 1
Unified Addressing: 1
----------------------------
```

1. Architecture and Compute Capability

The GPU used in this system is **NVIDIA GeForce GTX 1650**.
It has a compute capability of **7.5**, which corresponds to the **Turing architecture**.

2. Maximum Block Dimensions

The maximum block dimensions supported by the GPU are:
(1024, 1024, 64)

3. Maximum Number of Threads

Given:

- Maximum grid dimension = 65535
- Maximum block dimension = 512

$$\text{Total threads} = 65535 \times 512 = 33{,}553{,}920$$

The maximum number of threads that can be launched is **33,553,920**.

4. Why not always use maximum threads?

A programmer may choose not to use the maximum number of threads due to:

- Limited shared memory per block
- Register limitations
- Memory bandwidth constraints
- Thread divergence
- Inefficient workload distribution

5. What limits maximum threads?

The number of threads that can be launched is limited by:

- Available shared memory
- Number of registers per thread
- Hardware constraints of GPU
- Kernel resource requirements

6. Shared Memory

Shared memory is a fast on-chip memory accessible by threads within the same block.
It is used for efficient data sharing and reduces global memory access.

The shared memory available per block on this GPU is **48 KB**.

7. Global Memory

Global memory is a large but slower memory accessible by all threads on the GPU.

The total global memory available on this GPU is **4095 MB (~4 GB)**.

8. Constant Memory

Constant memory is a small read-only memory cached for faster access when all threads read the same data.

The constant memory available on this GPU is **64 KB**.

9. Warp Size

Warp size represents the number of threads executed simultaneously in a single instruction group.

The warp size of this GPU is **32**.

10. Double Precision Support

Double precision is supported on GPUs with compute capability ≥ 1.3.
Since this GPU has compute capability **7.5**,
double precision is supported.

Question: 2 Array Sum

```
bharti_vermaa@Bharti:~/UCS645/LAB6$ nano arraySum.cu
bharti_vermaa@Bharti:~/UCS645/LAB6$ nvcc arraySum.cu -o arraySum
bharti_vermaa@Bharti:~/UCS645/LAB6$ ./arraySum
Final Sum = 1024.000000
bharti_vermaa@Bharti:~/UCS645/LAB6$
```

Step 1: Allocate Device Memory

Device memory is allocated using cudaMalloc() for input and output arrays.

Step 2: Copy Host to Device

Data from host (CPU) is copied to device (GPU) using cudaMemcpy().

Step 3: Initialize Grid and Block

The kernel is launched with:

- 4 blocks
- 256 threads per block

Step 4: Invoke Kernel

The kernel is launched using:

sumKernel<<<blocks, threads>>>(...);

Step 5: Copy Device to Host

Partial results are copied back to host memory using cudaMemcpy().

Step 6: Free Device Memory

Allocated GPU memory is freed using cudaFree().

Step 7: CUDA Kernel Explanation

The kernel computes the sum using:

- Parallel computation
- Each thread processes multiple elements
- Shared memory for fast reduction
- Final block-wise sum stored in output array

Question: 3 Matrix Addition

Keeping n=4 to see matrices

```
bharti_vermaa@Bharti:~/UCS645/LAB6$ nano cudaMatrix.cu
bharti_vermaa@Bharti:~/UCS645/LAB6$ nvcc cudaMatrix.cu -o cudaMatrix
bharti_vermaa@Bharti:~/UCS645/LAB6$ ./cudaMatrix

Matrix A:
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1

Matrix B:
2 2 2 2
2 2 2 2
2 2 2 2
2 2 2 2

Matrix C (Result = A + B):
3 3 3 3
3 3 3 3
3 3 3 3
3 3 3 3
bharti_vermaa@Bharti:~/UCS645/LAB6$
```

Keeping n=1024

```
bharti_vermaa@Bharti:~/UCS645/LAB6$ nano cudaMatrix.cu
bharti_vermaa@Bharti:~/UCS645/LAB6$ nvcc cudaMatrix.cu -o cudaMatrix
bharti_vermaa@Bharti:~/UCS645/LAB6$ ./cudaMatrix
Sample Output C[0] = 3
Sample Output C[0] = 3
bharti_vermaa@Bharti:~/UCS645/LAB6$
```

1. Number of Floating Operations

In matrix addition, each element requires one addition operation.

Total elements = N^2

Therefore, total floating-point operations = $\mathbf{N^2}$

For N = 1024:

$$1024^2 = 1{,}048{,}576 \text{ operations}$$

2. Global Memory Reads

Each element requires:

- 1 read from matrix A
- 1 read from matrix B

Total reads per element = 2

Total global memory reads = **2 × N²**

For N = 1024:

$$2 \times 1{,}048{,}576 = 2{,}097{,}152 \text{ reads}$$

3. Global Memory Writes

Each element writes:

- 1 value to matrix C

Total writes per element = 1

Total global memory writes = **N²**

For N = 1024:

$$1{,}048{,}576 \text{ writes}$$